

Retrospective + Build-in-Public Post

AI Fluency Track — FL-10 Final Checkpoint

Retrospective (for the person I was in Week 1)

Week 1, you wrote a proof statement and had to defend it under pushback until it was actually narrow enough to be true. That turned out to be the whole track in miniature: not "can AI make something," but "can you tell, specifically, what part of the result is real."

What you set out to do was straightforward on paper — build a portfolio, ship a capstone, learn to work with AI properly. What actually happened is that almost every assignment quietly became the same exercise in a different costume: build something, then find out where it lies to you.

The ML capstone is the clearest version. FlyRank's own review flag was firing on 75-94% of pages in every tier — not a shortlist, nearly the whole inventory. You built a tier-adjusted model instead, and it looked good immediately: 0.60 precision against a 0.35 baseline. Except a naive train/test split gave an even better-looking 0.75 — and the only reason you know that number is fake is that you built the honest version too and compared them side by side. Two weeks later, a column-naming collision silently dropped your second-most-important feature out of the model with no error thrown. You caught that one too, but only because you'd gotten in the habit of checking, not because anything told you to look.

That habit is what actually changed, not any specific skill. By the time you got to the agent (FL-06/07), you ran the same system prompt twice and found it gave inconsistent answers — flagged a vendor's benchmark claim as unconfirmed once, stated an almost identical claim as fact the next time. Nobody told you to run it twice. You did it because by then "did it work" had quietly become "did it work the same way when I wasn't looking," and that's a different question with a different answer.

The portfolio crit was the one that stung the most, and probably taught the most. Amey's entire piece of feedback was one line — he couldn't tell what your main project actually was, in ten seconds, on a page you'd been staring at for weeks. You'd led with job titles instead of the actual thing you build. No amount of re-reading your own copy would have caught that, because you already knew what you meant. That's the specific, transferable version of "you can't see your own work clearly" — it's not a platitude, it's a blind spot with a shape, and someone else has to point at it.

Three things that actually transfer, not generic reflection:

First: validation isn't a step you add at the end, it's the difference between a result and a claim. The tier-adjusted model, the double-submit bug on the contact form, the agent's guardrail — all the same move: build the honest check before you trust the number.

Second: real feedback beats self-review every time, and defending against it defeats the entire point of asking. The crit only worked because you took "I didn't understand" as data, not as something to argue with.

Third: AI is a fast drafting partner and a genuinely bad judge of its own work. Every real catch in this track — the leaky split, the collision bug, the inconsistent guardrail, the confusing subheading — came from you checking, not from the tool flagging itself.

What you'd build next: the agent still only handles one topic manually-triggered; the natural next step is the full five-topic brief on a real schedule, which means finally dealing with the guardrail-inconsistency problem properly instead of just documenting it. And the ML capstone was validated on a 30,000-row starter slice — the real test is whether any of this holds at the actual 79-million-row warehouse scale.

You're not the person who wrote that first proof statement anymore. That person hoped the AI got it right. You now build the test that tells you when it didn't.

Build-in-public post

One real decision + one real limitation, ready to post:

Wrapping up an 8-week AI fluency capstone — a research agent that gathers, evaluates, and verifies AI/ML news before drafting a brief, instead of just trusting the first search result.

The decision I'm proudest of: instead of trusting a fixed retry rule, I let the model decide for itself when it has enough evidence — and when a claim needs independent verification before I'll believe it. When a source fetch got blocked by bot detection mid-run, it fell back to a second search on its own rather than giving up or reporting an unverified claim as fact.

The honest limitation: that same guardrail isn't 100% reliable. I ran the identical instructions twice — once it correctly flagged a vendor's benchmark claim as unconfirmed, the next run it stated a near-identical claim as settled fact. I found that by testing my own build twice with no reason to expect a problem, not because anything broke loudly. That inconsistency is still open, and it's in the README, not hidden in a footnote.

Repo / demo: [\[link\]](#)